Práctica guiada: plan de incrementos

Desarrollo de Aplicaciones y Servicios Virtuales · Nov 4, 2026

Propósito y punto de partida

Al terminar esta práctica, el equipo tendrá un plan de incrementos derivado por completo de su documento de especificaciones, y cada incremento desarrollado, probado y publicado en GitHub con su propio tag.

Resultado de aprendizaje: planear el desarrollo restante en incrementos ejecutables y trazables a la especificación, y gestionarlos con Git y GitHub como lo haría un equipo profesional.

Regla central: el documento de especificaciones es la única fuente del plan. Lo que no está en el documento no se planea: se registra como ambigüedad y el equipo lo resuelve con el profesor antes de continuar.

Antes de empezar, verifiquen que tienen:

- ☐ El documento de especificaciones aprobado (RF, RNF, historias de usuario, criterios de aceptación y arquitectura), en PDF o Word.
- ☐ El repositorio en GitHub con el incremento 1 publicado en la rama main.
- ☐ La práctica del primer módulo completa: su ciclo de 6 pasos se repite para cada tarea de cada incremento.
- ☐ Git configurado con acceso a GitHub (token de acceso personal o GitHub CLI).
- ☐ El documento de especificaciones guardado en la carpeta `docs/` del repositorio, con un commit propio.

Guardar la especificación en `docs/` hace que ChatGPT, Claude y el profesor trabajen siempre sobre la misma versión, y que cualquier cambio al documento quede registrado en el historial.

Cómo usar esta guía

Esta guía no es el plan: es el procedimiento para crearlo y después desarrollarlo. El plan lo produce cada equipo en la fase A, a partir de su propia especificación, y vive en su repositorio como `docs/plan_incrementos.md`. Las tablas de esta guía son el registro de sus decisiones y evidencias; solo la ficha por incremento de A3 pasa al plan.

A la IA no se le entrega esta guía. Recibe dos cosas:

- **El documento de especificaciones**, una sola vez al inicio: se sube al proyecto de ChatGPT y al de Claude, y queda disponible para todos los prompts.
- **El prompt del paso en curso**, con los corchetes completados.

Los prompts se envían uno a la vez y en orden. Después de cada respuesta, el equipo hace el punto de control antes de seguir: si un error del primer paso pasa sin revisar, todo lo que sigue se construye sobre él.

Paso	Herramienta	Qué se le pasa	Punto de control del equipo antes de seguir
A1	ChatGPT	Especificación y prompt A1	Los totales de requerimientos coinciden con su conteo
A2	ChatGPT, misma conversación	Prompt A2 y los RF del incremento 1	Cada dependencia tiene respaldo en el documento
A3	ChatGPT, misma conversación	Prompt A3 y sus restricciones	Aprueban, modifican o rechazan cada incremento
A4	ChatGPT, conversación nueva	Especificación, plan y prompt A4	Resuelven las observaciones del checklist
B1	Claude	Ficha del incremento, especificación, código actual y prompt B1	Aprueban la lista de tareas
B3	Claude	Prompts 2, 3 y 4 de la práctica anterior, una tarea a la vez	Entienden el código, las pruebas pasan, hacen commit y push
C1	Claude	Prompt C1	La regresión pasa
C2	ChatGPT	Resumen del incremento y prompt C2	Deciden qué hacer con cada observación

A1, A2 y A3 van en la misma conversación porque cada paso usa el resultado del anterior. A4 va en una conversación nueva a propósito, para que la revisión no esté influida por el razonamiento que generó el plan.

Roles y reglas de trabajo

Se mantiene la separación de funciones de la práctica anterior: ChatGPT analiza y documenta, Claude desarrolla y prueba, y el equipo decide y verifica.

Rol	Función en esta práctica	Lo que NO hace
Equipo de desarrollo	Aprueba el plan, autoriza cada tarea, ejecuta pruebas, hace commit y push, y verifica en GitHub	Aceptar código o un plan que no puede explicar
ChatGPT (análisis y documentación)	Lee la especificación, arma el inventario de requerimientos, propone los incrementos, valida el plan y contrasta lo construido con lo especificado	Generar código

Rol	Función en esta práctica	Lo que NO hace
Claude (desarrollo y pruebas)	Analiza el repositorio, planea las tareas de cada incremento, implementa, diseña y ejecuta pruebas	Cambiar el alcance del incremento o el plan aprobado

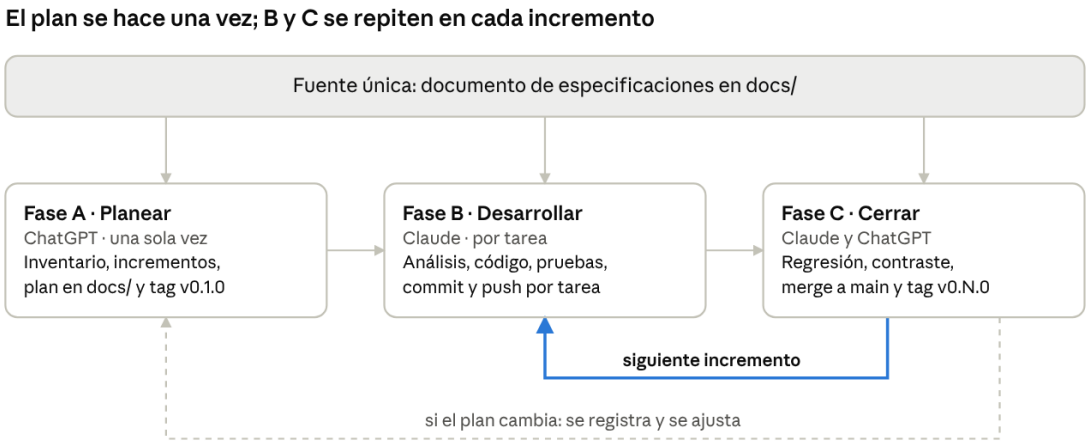
Reglas que se suman a las de la práctica del primer módulo:

Regla	Qué significa
Todo sale de la especificación	Cada RF del plan cita la sección del documento donde aparece. Sin cita, no entra al plan.
Incremento ejecutable	Cada incremento termina en algo que se puede usar de extremo a extremo, no en una capa técnica suelta.
Un commit por tarea	Dentro de un incremento hay varios commits pequeños, cada uno con su push. Nunca un solo commit con todo el incremento.
Un tag por incremento	El cierre de cada incremento se marca con un tag publicado en GitHub (`v0.2.0`, `v0.3.0`…).
Regresión obligatoria	Antes de cerrar un incremento se vuelven a ejecutar las pruebas de los incrementos anteriores.
El plan se versiona	El plan vive en `docs/plan_incrementos.md` del repositorio; cada cambio se justifica en el registro de cambios.

Por qué commits por tarea y no uno por incremento: un solo commit junta días de trabajo en un cambio enorme que no se puede revisar, revertir por partes ni rastrear cuando algo falla. El tag es lo que identifica al incremento completo.

Flujo general

El plan se construye una sola vez (fase A); después, cada incremento recorre las fases B y C hasta que no quedan requerimientos pendientes.



flujo de la práctica · 3 fases, B y C por incremento

Las tres fases leen la misma especificación. Si al cerrar un incremento el plan necesita ajustes, se registran y se regresa a la fase A solo para ese cambio.

Fase A. Planear los incrementos con ChatGPT

La fase A produce un solo entregable: `docs/plan_incrementos.md`, aprobado por el equipo y publicado en GitHub. Se hace una vez y se revisa al cerrar cada incremento.

A1. Cargar la especificación y verificar que se leyó completa

Creen un proyecto en ChatGPT para la materia, suban el documento de especificaciones y usen el prompt A1. En documentos largos el modelo puede omitir secciones; por eso el primer paso es comprobar que lo leyó todo antes de pedirle cualquier análisis.

Si su documento no numera los requerimientos, numérenlos ustedes en el documento (RF-01, RNF-01, HU-01…) y vuelvan a subirlo. Sin identificadores no hay trazabilidad.

Prompt A1 · ChatGPT: lectura y verificación del documento

```
Actúa como analista de requerimientos. Te adjunto el documento de especificaciones de
nuestro proyecto. Es la ÚNICA fuente de información: no agregues requerimientos,
supuestos ni funcionalidades que no aparezcan en él. Antes de cualquier análisis: (1)
lista los títulos de todas las secciones del documento; (2) lista todos los
identificadores de requerimientos funcionales, no funcionales, historias de usuario y
criterios de aceptación, con la sección donde aparece cada uno; (3) indica el total
de cada tipo; (4) señala los requerimientos sin identificador, sin criterio de
aceptación o con redacción ambigua. No propongas todavía ningún plan.
```

Cuenten ustedes los requerimientos en el documento y comparen. No continúen hasta que los totales coincidan; si no coinciden, pidan a ChatGPT que revise la sección donde está la diferencia.

Tipo	Total según ChatGPT	Total contado por el equipo	¿Coinciden?
Requerimientos funcionales (RF)	[Completar]	[Completar]	[Sí / No]
Requerimientos no funcionales (RNF)	[Completar]	[Completar]	[Sí / No]
Historias de usuario (HU)	[Completar]	[Completar]	[Sí / No]
Criterios de aceptación (CA)	[Completar]	[Completar]	[Sí / No]

Requerimiento	Ambigüedad detectada	Resolución del equipo
[Completar]	[Completar]	[Completar]
[Completar]	[Completar]	[Completar]

A2. Inventario de requerimientos pendientes

El inventario es la lista completa de lo que falta construir. Antes de usar el prompt, copien de la matriz de trazabilidad de la práctica anterior la lista de RF implementados en el incremento 1.

Prompt A2 · ChatGPT: inventario de requerimientos

```
Con base solo en el documento, construye una tabla con las columnas: ID, descripción breve (máximo 15 palabras), tipo, prioridad tal como la indica el documento (si no la indica, escribe "no indicada"), dependencias con otros requerimientos (solo las que se deducen del documento, citando la sección), sección del documento y estado. Los requerimientos implementados en el incremento 1 son: [lista de RF del incremento 1]; márcalos como "Implementado" y los demás como "Pendiente". No inventes prioridades ni dependencias.
```

ID	Descripción breve	Prioridad	Depende de	Sección del documento	Estado
[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Implementado / Pendiente]
[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Implementado / Pendiente]
[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Implementado / Pendiente]

Pregunta de control: ¿cada dependencia está respaldada por una sección del documento, o es una suposición del modelo? Eliminen las que no tengan respaldo.

A3. Propuesta de incrementos

Primero registren las restricciones reales del equipo; ChatGPT las necesita para dimensionar los incrementos.

Dato	Respuesta del equipo
Semanas disponibles hasta la entrega final	[Completar]
Número de incrementos que pueden cerrar	[Completar]
Integrantes del equipo	[Completar]

Prompt A3 · ChatGPT: plan de incrementos

```
Con el inventario aprobado, propón un plan de incrementos para los requerimientos
pendientes. Restricciones: tenemos [N] semanas, podemos cerrar [M] incrementos y
somos [K] integrantes. Aplica estos criterios en este orden: (1) un requerimiento va
después de aquellos de los que depende; (2) los de mayor prioridad según el documento
van primero; (3) lo de mayor riesgo técnico va en los primeros incrementos; (4) cada
incremento produce una funcionalidad ejecutable de extremo a extremo, integrada con
el incremento 1. Para cada incremento entrega: número y objetivo en una frase, RF e
HU incluidos, criterios de aceptación citados del documento, módulos involucrados
según la arquitectura del documento, dependencias con incrementos anteriores y
pruebas mínimas. Justifica cada agrupación. Usa únicamente información del documento;
si algo no se puede decidir con él, márcalo como pregunta para el equipo. No generes
código.
```

Completen una ficha por incremento. Esta ficha es lo que después le entregarán a Claude en la fase B.

Campo	Respuesta del equipo
Incremento y versión	[Incremento 2 · v0.2.0]
Objetivo en una frase	[Completar]
RF e HU incluidos	[Completar]
Criterios de aceptación (con sección del documento)	[Completar]
Módulos involucrados	[Completar]
Depende de	[Completar]
Pruebas mínimas: normal, límite y error	[Completar]
Rama de Git	[incremento-2]
Fecha de cierre	[Completar]
Decisión del equipo y justificación	[Aceptada / modificada / rechazada] · [Completar]

A4. Validación cruzada del plan

Abran una conversación nueva dentro del mismo proyecto de ChatGPT para esta revisión. Una conversación nueva revisa el plan sin el razonamiento que lo justificó, como lo haría un revisor externo.

Prompt A4 · ChatGPT: validación del plan contra la especificación

```
Revisa el plan de incrementos adjunto contra el documento de especificaciones.
Verifica: (1) que cada requerimiento pendiente aparece en exactamente un incremento;
(2) que ningún incremento incluye algo que no esté en el documento; (3) que ningún
requerimiento queda antes de uno del que depende; (4) que cada incremento termina en
una funcionalidad ejecutable; (5) que la carga es razonable para [N] semanas y [K]
integrantes. Devuelve un checklist con lo que cumple y lo que no, citando secciones
del documento. No reescribas el plan.
```

- ☐ Todos los requerimientos pendientes están asignados, y solo una vez.
- ☐ Ningún incremento incluye funcionalidades fuera del documento.
- ☐ El orden respeta todas las dependencias.
- ☐ Cada incremento termina en algo ejecutable de extremo a extremo.
- ☐ Las observaciones de la revisión se resolvieron o se justificó por qué no.

A5. Publicar el plan en GitHub

Pidan a ChatGPT el plan final en Markdown (inventario, una sección por incremento con su ficha y las ambigüedades resueltas) y guárdenlo como `docs/plan_incrementos.md`. Antes de publicarlo, marquen con un tag el estado actual de main: es el incremento 1 terminado.

```
git switch main
git pull
git tag -a v0.1.0 -m "Incremento 1: [objetivo]"
git push origin v0.1.0
git add docs/plan_incrementos.md
git commit -m "docs: plan de incrementos aprobado"
git push origin main
```

Opcional, pero recomendado: en GitHub creen un *Issue* por cada RF pendiente y un *Milestone* por incremento. Si un commit incluye `Closes #12`, GitHub enlaza ese commit con el requerimiento y cierra el issue automáticamente.

Evidencia	Dato
URL de `docs/plan_incrementos.md` en GitHub	[Completar]
Tag `v0.1.0` visible en GitHub (Tags)	[Sí / No]
Hash del commit del plan	[Completar]

Fase B. Desarrollar cada incremento con Claude

Cada incremento se desarrolla en su propia rama, tarea por tarea, y a cada tarea se le aplica el ciclo de la práctica del primer módulo: análisis, implementación, pruebas, commit y push.

B1. Cómo pasarle el incremento a Claude

Claude necesita tres cosas, y siempre en su versión actual:

1. La ficha aprobada del incremento (sección A3).
2. El documento de especificaciones, o al menos las secciones que cita la ficha.

3. El código actual del repositorio, tal como está en main.

Si usan Claude en el navegador: creen un proyecto y agreguen como conocimiento del proyecto la especificación, `docs/plan_incrementos.md` y los archivos de código. Al iniciar cada incremento, reemplacen los archivos de código por la versión actual de main.

Si usan Claude Code dentro del repositorio: los documentos ya están en `docs/`; basta con indicar las rutas en el prompt.

En ambos casos, la fuente de verdad es el repositorio, no lo que se dijo en conversaciones anteriores.

Prompt B1 · Claude: plan técnico del incremento

```
Actúa como desarrollador del proyecto. Te comparto la ficha aprobada del Incremento
[N], las secciones del documento de especificaciones que cita y el código actual del
repositorio. Antes de programar: (1) revisa cómo está construido el código actual y
qué del incremento anterior se reutiliza; (2) descompón el incremento en tareas
pequeñas, cada una implementable y probable por separado, en el orden en que deben
hacerse; (3) para cada tarea indica el RF que atiende, los archivos que crearías o
modificarías y la prueba que demostraría que funciona; (4) señala ambigüedades o
contradicciones entre la ficha, la especificación y el código. No escribas código
todavía y no agregues tareas fuera de la ficha. Espera la autorización del equipo.
```

#	Tarea	RF	Archivos	Prueba que la demuestra	Decisión del equipo
1	[Completar]	[Completar]	[Completar]	[Completar]	[Aceptada / modificada / rechazada]
2	[Completar]	[Completar]	[Completar]	[Completar]	[Aceptada / modificada / rechazada]
3	[Completar]	[Completar]	[Completar]	[Completar]	[Aceptada / modificada / rechazada]

Si Claude detecta una contradicción con la especificación, no la resuelvan en el código: regresen a ChatGPT o al profesor, y registren la decisión en el registro de cambios al plan.

B2. Crear la rama del incremento

```
git switch main
git pull
git switch -c incremento-2
git push -u origin incremento-2
```

B3. Ciclo por tarea

Para cada tarea aprobada en B1, en orden:

1. **Análisis previo** con el Prompt 2 de la práctica del primer módulo, indicando el número y nombre de la tarea.
2. **Implementación** con el Prompt 3. Cada integrante debe poder explicar el código antes de aceptarlo.
3. **Pruebas** normal, límite y error con el Prompt 4, registrando el resultado real obtenido.

4. **Commit y push** de esa tarea, y verificación en GitHub.

Cada mensaje de commit indica el tipo de cambio y el requerimiento que atiende: `tipo(RF-XX): descripción`.

Tipo	Cuándo se usa	Ejemplo
feat	Funcionalidad nueva	`feat(RF-05): validar formato de correo en el registro`
test	Pruebas nuevas o ajustadas	`test(RF-05): casos de correo válido, vacío y mal formado`
fix	Corrección de un error	`fix(RF-03): corregir total cuando hay descuento`
refactor	Mejora interna sin cambiar el comportamiento	`refactor(RF-02): separar validación del controlador`
docs	Documentación	`docs: actualizar README con el incremento 2`

```
git status
git add <archivos de la tarea>
git commit -m "feat(RF-05): validar formato de correo en el registro"
git push
```

Agreguen los archivos por nombre en lugar de usar `git add .`, y confirmen que `.env` aparece en `.gitignore`. Así no se suben claves ni archivos temporales por accidente.

Tarea	RF	Hash del commit	Pruebas (pasa / falla)	Push verificado en GitHub
[Completar]	[Completar]	[Completar]	[Completar]	[Sí / No]
[Completar]	[Completar]	[Completar]	[Completar]	[Sí / No]
[Completar]	[Completar]	[Completar]	[Completar]	[Sí / No]

Fase C. Cerrar el incremento

Un incremento está cerrado cuando pasan sus pruebas y las de los incrementos anteriores, ChatGPT confirma que cumple la especificación y su tag está publicado en GitHub.

C1. Pruebas de regresión con Claude

Lo nuevo no debe romper lo que ya funcionaba. Antes de integrar, se vuelven a ejecutar todas las pruebas del proyecto.

Prompt C1 · Claude: regresión del proyecto

```
El Incremento [N] está implementado en la rama incremento-[N]. Ejecuta todas las pruebas del proyecto, incluidas las de los incrementos anteriores, e informa el resultado real de cada una. Si alguna prueba anterior falla, identifica qué cambio del incremento actual la rompió y propón la corrección, sin aplicarla hasta que el equipo la autorice. No afirmes que una prueba pasó si no se ejecutó.
```

Si Claude no puede ejecutar las pruebas en su entorno, las ejecuta el equipo y le pega la salida para que la analice.

Prueba	Incremento de origen	Resultado real	Estado
[Completar]	[Completar]	[Completar]	[Pasa / falla]
[Completar]	[Completar]	[Completar]	[Pasa / falla]
[Completar]	[Completar]	[Completar]	[Pasa / falla]

C2. Contraste final con ChatGPT

Prompt C2 · ChatGPT: cumplimiento del incremento

```
Revisa el resumen del Incremento [N] (tareas, commits y resultados de pruebas) contra su ficha en el plan y contra el documento de especificaciones. Identifica criterios de aceptación no cumplidos, diferencias con la arquitectura del documento y pendientes para la definición de terminado. Indica también si algo del plan de los incrementos siguientes debe ajustarse por lo aprendido en este. No generes código ni inventes evidencias. Devuelve un checklist priorizado.
```

Observación de ChatGPT	Decisión del equipo	Acción o justificación
[Completar]	[Atender ahora / pasar al siguiente incremento / descartar]	[Completar]
[Completar]	[Atender ahora / pasar al siguiente incremento / descartar]	[Completar]

C3. Integrar a main y publicar el tag

```
git switch main
git pull
git merge incremento-2
# Vuelvan a ejecutar las pruebas en main antes de publicar
git push origin main
git tag -a v0.2.0 -m "Incremento 2: [objetivo]"
git push origin v0.2.0
```

Alternativa recomendada para equipos: en lugar del merge local, abran en GitHub un *Pull Request* de `incremento-2` hacia `main`, y que otro integrante lo revise y apruebe. Después creen el tag sobre main.

Verifiquen en GitHub que el tag aparece en la sección *Tags*. Opcionalmente, creen un *Release* a partir del tag con un resumen de lo que entrega el incremento.

Definición de terminado del incremento

Amplía la definición de terminado de la práctica del primer módulo:

- ☐ Todos los RF del incremento cumplen sus criterios de aceptación.
- ☐ Cada tarea tiene su commit, con push verificado en GitHub.
- ☐ Las pruebas del incremento (normal, límite y error) pasan, con resultado real registrado.
- ☐ Las pruebas de los incrementos anteriores siguen pasando.
- ☐ El contraste con ChatGPT no deja pendientes críticos sin resolver.
- ☐ La rama está integrada a main y el tag del incremento está publicado.
- ☐ README, matriz de trazabilidad, bitácora de IA y plan están actualizados.

Evidencia	Dato
URL del tag en GitHub	[Completar]
Hash del commit de integración en main	[Completar]
URL del Pull Request (si aplica)	[Completar]
Pruebas de regresión: total y fallidas	[Completar]

Al terminar, revisen si el plan necesita ajustes y repitan las fases B y C con el siguiente incremento.

Registros acumulados

Estos tres registros se actualizan al cerrar cada tarea o incremento, y se entregan completos al final del proyecto.

Matriz de trazabilidad

Cada fila une un requerimiento con el lugar del documento de donde sale, el código que lo implementa y la prueba que lo demuestra. Incluyan también los RF del incremento 1.

RF / HU	Sección del documento	Incremento	Módulo o archivos	Commits	Tag	Prueba	Estado
[RF-01]	[Completar]	[1]	[Completar]	[Completar]	[v0.1.0]	[Completar]	[Terminado]
[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Pendiente / En curso / Terminado]
[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Completar]	[Pendiente / En curso /

RF / HU	Sección del documento	Incremento	Módulo o archivos	Commits	Tag	Prueba	Estado
							Terminado]

Registro de cambios al plan

El plan puede cambiar; lo que no puede pasar es que cambie sin registro. Todo cambio cita su motivo, y si contradice la especificación, requiere el visto bueno del profesor.

Fecha	Cambio al plan	Motivo	Aprobado por
[Completar]	[Completar]	[Completar]	[Completar]
[Completar]	[Completar]	[Completar]	[Completar]

Bitácora de IA

Fecha	Herramienta	Prompt o tarea	Qué se aceptó	Qué se modificó o rechazó, y por qué
[Completar]	[ChatGPT / Claude]	[Completar]	[Completar]	[Completar]
[Completar]	[ChatGPT / Claude]	[Completar]	[Completar]	[Completar]

Entrega y reflexión individual

Hay dos entregas: el plan, al terminar la fase A, y cada incremento, al terminar su fase C.

Entrega	Qué se entrega
Plan de incrementos	URL de `docs/plan_incrementos.md` en GitHub, tag `v0.1.0`, tablas de A1 a A4 completas
Cada incremento	URL del tag del incremento, tabla de tareas con hashes, resultados de pruebas y regresión, checklist de terminado y registros actualizados

Si un incremento no se completa, registren de forma explícita qué se logró, qué quedó pendiente y en qué incremento se atenderá, y reflejen el cambio en el registro de cambios al plan.

Integrante	¿Qué tareas del incremento puede explicar?	¿Qué diferencia encontró entre lo planeado y lo construido, y por qué?
[Nombre]	[Respuesta individual]	[Respuesta individual]
[Nombre]	[Respuesta individual]	[Respuesta individual]
[Nombre, si aplica]	[Respuesta individual]	[Respuesta individual]